

A Quorum-Based Total Order Broadcast Protocol for Distributed Intelligence Databases

Dalla Croce Enrico, Dematté Luca

Distributed Systems 1 Project – July 26, 2026

1 Project Structure

1.1 Subpackages

We created a subpackage named `cs` located at `it.unitn.ds.cs`. The code related to our abstractions is located under the `cs` package. Instead of placing message classes inside the class of the receiver (as suggested in the labs), we put all message classes in a subpackage `it.unitn.ds.cs.messages`. Inside this package, messages are divided based on who is the sender.

1.2 Naming Convention

To easily identify our classes all class names have the prefix `CS`.

2 Design

To keep the explanation concise, we focus solely on the aspects that complement the project instructions: our assumptions, additional features, and custom solutions.

2.1 Assumptions

2.1.1 Project Instructions

- Communication channels are assumed to be reliable and FIFO
- A strict majority of replicas never crashes.
- Replicas fail by crashing and do not recover.
- Static system membership: no new replicas may join after initialization.

2.1.2 UUID uniqueness

We assume that the generated UUIDs are unique. Replicas independently generate UUIDs and so technically it can happen that two independent UUIDs match, making them not unique.

It has to be noted that we rely on the `UUID` class included in the Java SDK which offers some guarantees.

The documentation for the method `public static UUID randomUUID()` states: "The UUID is generated using a cryptographically strong pseudo random number generator."

We can also look at the problem from a statistical point of view. A UUID represents a 128bit value, the chance of two random UUID being identical is $\frac{1}{2^{128}} = \sim 3 \times 10^{-39}$, which is negligible.

2.1.3 Replica Crash Detection

Since channels are reliable, if a replica does not Ack messages for which an Ack is expected (such as Update or Election messages), that replica can be considered crashed.

2.1.4 Timeout values

The timeouts values are all based on `AbstractReplica.getMaxLatencyPlusTolerance()` which internally takes into account the network latency and the number of active replicas. Through trial and error we found values that made the system responsive and avoided false positives. Additionally, by calculating timeouts based on such parameters rather than relying on constant values, the system should be able to adapt to larger groups of replicas.

2.2 AskSystem

We initially attempted to follow Akka Ask pattern using the provided `ask()` method, which returns a `CompletionStage` (future) that completes when a response is received. However, the future is completed on whatever thread delivers the response which is not guaranteed to be the requesting sending actor thread. Mutating actor state inside the callback introduce race conditions. Akka ask documentation: *"do not call methods or access mutable state [...] from within the callback. This [...] may introduce synchronization bugs and race conditions. [...] there is not yet a way to detect these illegal accesses at compile time."*

The standard solution is to use `pipeToSelf()`, which delivers the result as a message to the actor mailbox, where it is processed like any other message. But the project requires to use the `tell()` method of the codebase that emulates network latency. And this cannot be done using Akka own methods.

Therefore we designed our own wrapper called `askSystem` built on top of the custom `tell()` to emulate the ask pattern which makes very easy handling timeouts. `AskSystem` enables an actor to send messages and register a callback to be executed when a response is received and/or when the specified timeout expires. All messages sent through `AskSystem` must extend `CSAskMessage` which has an UUID as an attribute used to correlate a request with a response and trigger the correct callback.

When a message is sent a `CSAskTimeout` to self is scheduled through a timer. If the response is received in time the timer is canceled and the callback fired, otherwise if the `CSAskTimeout` message is received the callback is still fired but with the information that the timer expired.

Example: in Figure 1, the UUIDs used by the `AskSystem` are shown as superscript and color coded for each message.

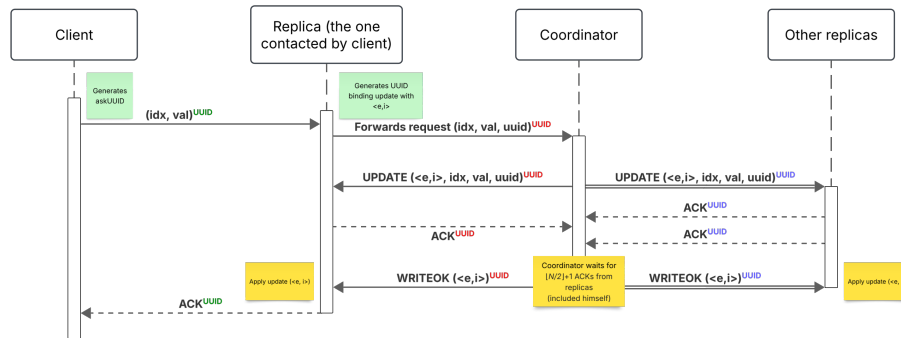


Figure 1: Update Protocol Diagram

2.3 Update Protocol

For write requests, we designed replicas so that they immediately keep track of the request in their history instead of logging it only when the Update message is received. This way, if the coordinator crashes before processing the request, the replica which forwarded it still has track of it and can try again after the election. This choice introduces a problem: replicas need to be able to correlate the request forwarded to the coordinator with the Update message they will receive later.

Example:

- ReplicaA and replicaB send WriteRequest (index=2, value=7)
- Coordinator sends Update (<0, 1>, index=2, value=7)
- Coordinator sends Update (<0, 2>, index=2, value=7)

Problem: there is now way of matching an update to the correct request

Solution: each replica appends to the Request message a privately generated UUID. The coordinator appends to the Update message the UUID contained in the request. When an Update message is received each replica checks for a matching UUID which binds the Update to its Request.

2.3.1 Crash Detection

During the update protocol the coordinator can detect crashed replicas that do not answer with an Ack.

The coordinator after detecting a crash removes the replica from its list of active replicas and sends a `CSCrashNotice` message to the previous replica so that it can update its own list of active replicas and the pointer to the next replica.

Originally crashed replicas are discovered only during the Election process when passing the message through the ring. This extra measure speeds up the Election process avoiding some unnecessary timeouts as crashed replicas can be discovered before the Election.

2.4 Updates Logging

Update logging is taken care by the class `CSUpdateLogger`. Every replica has an attribute `updateLogger` which is used to:

- Log write requests coming from clients
- Log updates coming from the coordinator

As discussed in section 2.3, it is important to log requests from the moment they are received from the client, as during the coordinator recovery (subsection 2.6.3) we process not only the updates that did not complete, but also the requests that have not yet been fulfilled.

As will be discussed further in section 3.2, the class enables updates to be identified in two ways:

- The UUID generated by the replica when the request of the client is received.
- The update key `<e,i>` that the coordinator uses in the update protocol.

2.5 Coordinator

The coordinator acts differently from replicas in almost every operation. We decided to introduce custom behavior rather than making it behave similarly to replicas, as this would have required sending messages to itself.

As a result all local operations are performed immediately, skipping the sending of any message to itself.

2.5.1 Heartbeat

The heartbeat detection is not done through the `AskSystem` which works only during update processing. We instead opted for a timer based system that in contrast works also during idle periods.

When a replica becomes a coordinator a recurring message to self is scheduled, which upon arriving triggers the broadcast of the heartbeat message to all replicas. Each replica schedules a recurring message to self too, which triggers a check that the heartbeat has been received during the interval.

A flag is raised when a heartbeat is received and lowered after every check. If the flag is not found raised during a check, an election is initiated.

The replicas start checking for a heartbeat with some delay to allow the coordinator to spin up. In addition the checking interval is a little longer than the beat interval to allow for slowdowns in the communication and processing. Everything combined absorbs propagation delay and avoids false positives.

2.6 Election

The ring structure needed during the election is maintained through a series of pointers. Each replica has an attribute `next` which stores the ID of the following replica in the ring.

2.6.1 Crash detection

When forwarding the election message through the ring if the timeout for an Ack expires the replica knows that its `next` has crashed so it can remove it from its list of active replicas and update its `next`

2.6.2 Non Blocking

To make sure that the election process terminates we do four things:

1. To avoid the circulation of duplicated Election messages replicas consider only the message sent by the replica with the highest ID. Election messages carry a value called `initiatorId`, and replicas keep track of the `initiatorId` of the message they are currently considering. When any election message is received, a replica drops it if the `initiatorId` is lower than the tracked one (Figure 2).
2. If a crash is detected the replica tries to contact the following active replica. The attempts continue indefinitely until a replica correctly sends back an Ack.
3. When passing the election message with the chosen coordinator, if the previous replica in the ring detects the crash of the winner it removes it from the election message, computes the new winner and continues to circulate the message.
4. As a catch-all solution for any other case every replica starts a timer at the start of every election. If the timer expires the replica starts a new election attempt. In a similar fashion to point 1. only the most recent attempt is considered (Figure 2).

```

if(
  msg.getElectionAttempt() <= this.electionAttempt &&
  msg.getInitiatorId() < this.electionInitiatorId) {
  //drop election message
} else {
  //accept election message
}

```

Figure 2: Condition to drop an election message

2.6.3 Recovery

After sending the synchronization message the coordinator starts the recovery of incomplete updates.

Step 1: compute oldest update completed across all replicas

Step 2: send incomplete updates

```

foreach update ∈ incompleteUpdates do
  | Run the update protocol for update
end

```

Step 3: handle write request received during the election

```

foreach writeRequest ∈ writeRequests do
  | update ← create update with a key in the new epoch for writeRequest
  | Run the update protocol for update
end

```

3 Implementation

3.1 Custom ask implementation

The core of the AskSystem is the `ask()` function that emulates the Akka ask pattern. The callback is defined with a functional interface (Figure 4) used to mimic ask original behavior (Figure 3).

Figure 6 shows an example of `ask()` in use. the return value type is parametric and so casting is automatically performed, `res` will be of the type specified.

```

CompletionStage result = AskPattern.ask(actor, message, timeout);
result.whenComplete(
  (reply, failure) -> {
    if(...) {...}
    else {...}
  });

```

Figure 3: ask in Akka Classic

```
@FunctionalInterface
public interface Callback<T extends CSAskMessage> {
    void handle(T response, boolean timedOut);
}
```

Figure 4: ask callback parameter type

```
public <T extends CSAskMessage> void ask(
    CSAskMessage msg, ActorRef destination,
    Duration timeout, Callback<T> callback
)
```

Figure 5: ask firm

```
askSystem.<CSUpdate>ask(new CSWriteForward(...),
    replicas.get(this.coordinatorId),
    timeout,
    (res, timedOut) -> {
        if (!timedOut) {...}
        else {...}
    })
```

Figure 6: ask usage example

3.1.1 WriteOk Timeout

When a replica sends (through `ask()`) its Ack for an update, a timer starts. To avoid the timer from expiring we need to send a WriteOk containing the correct askUUID to the replica. The problem is that the coordinator must broadcast the WriteOk message when the quorum is reached.

Our solution instead is for each replica to keep a map of the update key and the askUUID used to send the Ack. In `HandleUpdate()` the mapping is recorded in `Map<CSUpdateKey, UUID> updatesAskUUIDs`. Then when the WriteOk message is received it is not feed directly to `askSystem.handleResponse()`, previously a new WriteOk message is created with the correct askUUID retrieved from the map.

```
public final void handleWriteOk(CSWriteOk msg) {
    UUID askUUID = this.updatesAskUUIDs.get(msg.getKey());
    this.askSystem.handleResponse(new CSWriteOk(msg.getKey(), askUUID));
}
```

3.2 Update Logging

We leverage two different HashMaps:

- `Map<UUID, CSUpdateData> updates`
- `Map<CSUpdateKey, UUID> keyToUUIDBindings`

When a replica receives a WriteRequest from the client the coordinator has not assigned a key yet. The only identifying information is the generated UUID, so the update is first inserted into `updates`.

When replica receives an Update message it binds the key to the UUID through `keyToUUIDBindings`.

Upon receiving the WriteOk, which identifies the update only by its CSUpdateKey, we need to gather the update data chaining the two HashMaps: `updates.get(keyToUUIDBindings.get(key))`. The data contains information on the write to apply (index and value) and the reference to the client that is waiting for the WriteResult.

4 Information about the use of LLM tools

In this section, we report how we made use of the functionalities provided by LLM tools, as required by the updated project guidelines:

- Gemini: questions about Java.
- ChatGPT: (before starting with the development) questions about the codebase to better understand how to use it.
- OpenCode (Big Pickle): reasoning on the codebase and providing pseudo code.
- Claude, Mistral: help in writing the report. Used to write drafts.